Version Control

Date 15/09/26 Pranav Tuppada Srivardhan

Version Control

Version control (also called source control) is a system that records changes to a file or a set of files over time, so that specific versions can be recalled later. It is used mostly to track changes in the source code with a structured project history. It allows multiple developers to collaborate on the same project without overwriting each others work.
It records the history of the project and every time a change is made , that is recorded as a commit or revision based on the system. Instead of saving a new copy of a file every time you make a change, version control is beneficial in this case.

Version control systems usually offer you the ability to:

- Identify differences between any two versions of code
- Roll back files (or even projects) to an older version
- Determine who made a particular change and when it was made
- Develop several versions of a project at once without one overwriting the other
- Consolidate changes from different developers (or even from different changes from the same developer) into a consolidated version

Why is Version Control Important

Collaboration: It allows multiple people to work on the same document and they all can work without overwriting another persons changes. It provides tools to merge everyones changes together, flagging conflicts only when the same lines of code were changed in incompatible ways.

History and Reverting: Each and every commit is a milestone. If a change brings about an error, that causes the build to break, or turn out to be a mistake in itself, it will always be possible to revert to a previous, functioning version rather than trying to reverse the damage done.

Branching: By using version control, developers are able to make "branches" which refer to separate versions of the codebase that are independent from each other where they can work on

developing new features or experiments without altering the main version of the project. Once done the feature can then be merged back into the main branch.

Backup: The version history of a project is saved on a remote server (like GitHub) the project is already backed up. The entire project and all the version histories can still be cloned and retrieved even in cases where the developer loses his laptop or it gets stolen or corrupted.

Revision or Commit: Each revision is always linked with an author and time stamp and always carries a message explaining the change. The log trail allows one to not only see the change made, but also to comprehend the reasoning behind the change, thus allowing a developer to track down the reason for a bug created weeks ago.

Continuous Integration: Version control systems integrate directly with automation tools. Every time you push new code, automated workflows can instantly test, build, and deploy the software, reducing manual work and errors.

Types of Version Control

There are three main types of version control and they are as follows:

1. Local VCS
2. Centralized VCS
3. Distributed VCS

1.Local Version Control

Keeps track of file changes on a single local computer, usually by copying entire files into numbered or dated folders, or by using a simple local database of patches (differences between file versions).

Commands like committing, viewing history, and switching branches happen instantly because the program doesn't have to ping a remote cloud server.

No Collaboration,sharing project states with another person becomes an awkward process of manual file exports.

2.Centralised Version Control

Centralized version control systems introduced a single central server that stores the complete version history of the project.

Programmers will download files to their local computer and do modifications on them, which are subsequently uploaded to this central repository. There exists one single definitive version of

the history of the project and that lies in the central server. Each user will have just one working version of the current version in his local machine.

Easier to understand and administer than distributed systems; everyone has visibility into what everyone else is doing, since all commits go through the same server.

The main server is the single point of failure if it crashes or the information contained in it is lost, without proper backup, all the projects history will be lost. Many processes depend on having access to the server through the network.

3.Distributed Version Control

Every contributor has a complete, local copy of the code repository and its entire history. The cloning of a repository by the programmer does not only allow him access to the current files but to the full history of the project. Committing is done on the local machine, and then the commit is pushed or pulled to a remote repository.

Git is an example of distributed version control

Advantage:

- Can work fully offline, since the entire history is available locally
- No single point of failure every clone is effectively a full backup of the project

Disadvantage: Takes time to learn concepts

Important Git Commands

1. git innit- initialises a new repository
2. git clone<repository>-creates a copy of (or downloads) an existing repository
3. git status-shows the present status of the working directory
4. git add <filename>- stages a specific files change, git add . stages all changed files in the current directory
5. git diff- shows the exact line-by-line differences between the working directory and the last commit
6. git commit -m “message”- saves all staged changes as a new commit, along with a descriptive message explaining what was changed
7. git log- displays the commit history, useful for reviewing the evolution of a project

8. git branch- lists all local branches in your repository
9. git switch <branch-name>- switches to a different branch
10. git merge <branch-name>- combines changes from a specified branch into the active one
11. git push origin <branch-name>- uploads your local commits to the remote repository
12. git fetch- downloads history and updates from the remote repository
13. git pull- Fetches and automatically merges remote changes into your current local branch

Bibliography

Atlassian. (n.d.). What is version control? Atlassian Git Tutorials. Retrieved September 16, 2026, from https://www.atlassian.com/git/tutorials/what-is-version-control

Coursera. (n.d.). What is version control? How it works. Coursera Articles. Retrieved September 16, 2026, from https://www.coursera.org/articles/version-control

GitHub Education. (n.d.). Git cheat sheet [PDF]. https://education.github.com/git-cheat-sheet-education.pdf

GeeksforGeeks. (n.d.). Version control systems. Retrieved September 16, 2026, from https://www.geeksforgeeks.org/git/version-control-systems/

Wikipedia contributors. (n.d.). Distributed version control. In Wikipedia, The Free Encyclopedia. Retrieved September 16, 2026, from https://en.wikipedia.org/wiki/Distributed_version_control